

McGill Course Advisor Agent

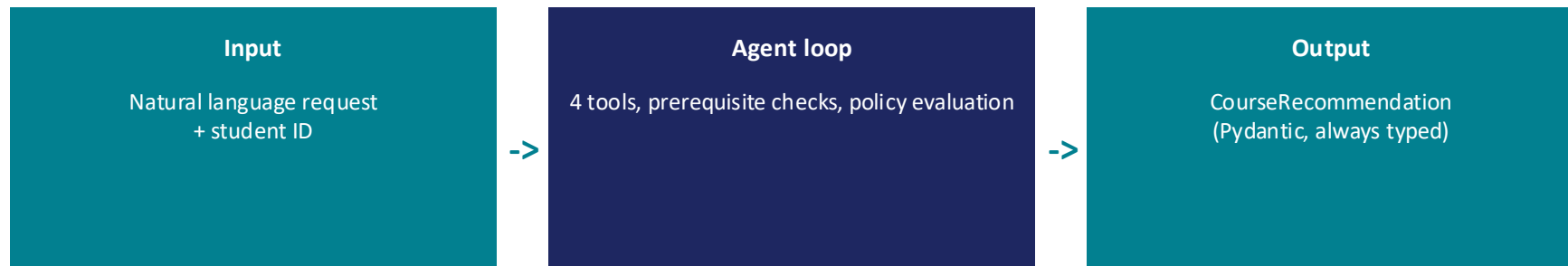
Single-Agent AI System using the OpenAI Agents SDK

MGSC 697 | Python | gpt-4o | 10/10 evals passing

By Laura Manzanos (ID: 261284603)

What does it do?

A student types a natural language request. The agent looks up their profile, searches a fake McGill course catalog, checks prerequisites, flags policy violations, and returns a structured recommendation including whether a human advisor needs to approve.



One agent. No multi-agent system. No production deployment.

4

typed tools

1

named guardrail

12

courses

5

student profiles

10/10

evals passing

The Fixture World (fixtures.py)

COURSES (12 total)

INFO201	Programming for Business	200-level, no prereqs
MGSC301	Intro to Analytics	300-level, no prereqs
MGSC310	Database Management	300-level, no prereqs
FINE301 / MGMT301	Finance + Ethics	300-level, no prereqs
MGSC401	Predictive Analytics	400-level, needs MGSC301
MGSC415	Machine Learning	400-level, needs MGSC401
MGSC430	Big Data and Cloud	400-level, needs MGSC310
FINE410	Quantitative Finance	400-level, 0 seats (approval req.)
MGSC499	Capstone (6 cr)	400-level, GPA 3.0+ required (approval req.)

STUDENTS (5 profiles)

S001	Alice	GPA 3.7, good standing, mid-program
S002	Ben	GPA 1.6, probation, 3 cr enrolled
S003	Chloe	GPA 3.2, good, near graduation
S004	Diego	GPA 1.8, probation, has capstone prereqs
S005	Emma	GPA 3.5, good, first semester

POLICY RULES

- Max 18 credits/semester (all students)
- Max 12 credits/semester on probation (approval if over)
- GPA 3.0+ required for MGSC499 (approval if below)
- 0 seats on FINE410 (advisor approval for waitlist)

How Every Request Works

0

PROBATION_OVERLOAD_GUARDRAIL

Runs before the agent loop. If standing=probation AND credits >= 12, block immediately. No tool calls, no API cost.

1

get_student_profile

Always the first tool call. Loads GPA, standing, and completed courses. Only source of truth about the student.

2

search_courses

Searches catalog by keyword tokens. Agent can only recommend courses that appear here. Hallucinating course codes is prevented.

3

check_prerequisites

One call per candidate course. Returns eligible, missing prerequisites, already completed. Ineligible courses are dropped.

4

flag_policy_risk + structured output

Checks credit cap, GPA minimum, seat availability. Any flag sets requires_advisor_approval=True. Returns CourseRecommendation

Key Code Snippets

Agent definition

```
advisor_agent = Agent(
    name="McGill Course Advisor",
    model="gpt-4o",
    output_type=CourseRecommendation,
    tools=[get_student_profile,
           search_courses, check_prerequisites,
           flag_policy_risk],
    input_guardrails=[probation_guardrail])
```

Example tool

```
@function_tool
def check_prerequisites(
    ctx: RunContextWrapper[AdvisorContext],
    course_code: str,
) -> dict:
    course = COURSES.get(course_code.upper())
    completed = set(student['completed_courses'])
    missing = [p for p in course['prerequisites']
               if p not in completed]
    return {
        "eligible": len(missing) == 0,
        "missing_prerequisites": missing }
```

Named guardrail

```
@input_guardrail
async def probation_credit_guardrail(
    ctx: RunContextWrapper[AdvisorContext],
    agent: Agent, input: str
) -> GuardrailFunctionOutput:
    student = STUDENTS.get(ctx.context.student_id)
    on_probation = student['standing'] == 'probation'
    at_cap = student['credits_this_semester'] >= 12
    return GuardrailFunctionOutput(
        tripwire_triggered=on_probation and at_cap)
```

Run helper (state strategy)

```
async def advise(student_id: str, message: str):
    # student_id lives in context, not the prompt
    context = AdvisorContext(student_id=student_id)
    try:
        result = await Runner.run(
            advisor_agent, message,
            context=context, max_turns=30)
    except InputGuardrailTripwireTriggered:
        raise RuntimeError('[GUARDRAIL TRIGGERED]')
```

Agent Design

4 typed @function_tool decorators

Tool	Input	Returns
<code>get_student_profile</code>	none (uses context)	GPA, standing, completed courses
<code>search_courses</code>	query, dept?, level?	Matching course list
<code>check_prerequisites</code>	course_code	eligible + missing prereqs
<code>flag_policy_risk</code>	course_codes[]	Policy flags + approval signal

Guardrail: PROBATION_OVERLOAD_GUARDRAIL

Type: @input_guardrail -- runs before the agent loop starts

Condition: student.standing == probation AND credits_this_semester >= 12

Effect: tripwire_triggered=True -- RuntimeError raised, no tool calls made

Why input?: Enrollment is irreversible. Block early = zero API cost + no false recommendation

State strategy

Student identity	AdvisorContext.student_id (context=)
Profile data	Fetches by tool, recomputed each run
Conversation	Runner in-memory session
Tool results	Deterministic from fixtures

Structured output: CourseRecommendation

```
class CourseRecommendation(BaseModel):  
  
    recommended_courses: list[CourseSlot]  
  
    risks: list[str]  
  
    policy_flags: list[PolicyFlag]  
  
    requires_advisor_approval: bool  
  
    approval_reason: Optional[str]  
  
    total_recommended_credits: int  
  
    summary: str
```

ID	Student	Scenario	Type	Pass if...
EVAL-01	S001	Requests ML/analytics courses	Happy path	MGSC401 recommended, no approval
EVAL-02	S001	Requests MGSC415 (missing prereq)	Failure	MGSC415 excluded, risks populated
EVAL-03	S002	Probation at 12-credit cap	Guardrail fires	Request blocked before agent runs
EVAL-04	S003	Requests capstone, GPA 3.2 >= 3.0	Policy boundary	MGSC499 recommended, no approval
EVAL-05	S003	Requests already-completed MGSC301	Edge	MGSC301 not in recommendations
EVAL-06	S001	Off-catalog query (neuroscience)	Edge / empty	Empty output, risks explain
EVAL-07	S004	Capstone, GPA 1.8 < 3.0 minimum	Approval required	requires_advisor_approval=True + GPA flag
EVAL-08	S002	Probation, 3 credits (below cap)	Guardrail boundary	Guardrail does NOT fire
EVAL-09	S001	One eligible + one ineligible course	Mixed	Only eligible recommended
EVAL-10	S001	FINE410, 0 seats available	Approval required	requires_advisor_approval=True + seat flag

What Failed and What We Fixed

MaxTurnsExceeded (default limit: 10 turns)

Fix: Agent needs 7+ turns minimum. Raised max_turns to 30.

MaxTurnsExceeded again at 30 turns

Fix: gpt-4o-mini called search_courses twice. Switched to gpt-4o.

search_courses returned empty for multi-word queries

Fix: Was matching full phrase. Fixed: split into tokens, match any word.

Agent recommended already-completed courses

Fix: System prompt did not prohibit it. Added explicit rule and EVAL-05.

Guardrail predicate off by one (> 12 not >= 12)

Fix: Student at exactly 12 credits was not blocked. Fixed to >=. Added EVAL-08.

EVAL-07 used wrong student (S002 lacked prereqs)

Fix: GPA check never triggered. Added S004 who has all capstone prereqs.

Agent hallucinated courses when search returned empty

Fix: Added rule: only recommend courses returned by search_courses.

Reflection

What I set out to build

A course advisor agent that takes a student request in plain language, grounds every decision in deterministic fixture data (not model memory), and returns a typed structured output with an honest risk list and a machine-readable approval flag. Narrow scope done correctly.

What worked

Typed tools eliminated hallucination of facts. Defining CourseRecommendation as a Pydantic model before writing any agent code forced every design decision to be explicit. Placing the guardrail at the input layer meant blocking happened before any API cost was incurred.

What is still risky

Tool call sequence is enforced by system prompt only -- code cannot prevent the model from skipping steps. The agent only works reliably with gpt-4o; swapping to a cheaper model silently breaks it. Only one hard guardrail exists. Each additional risk scenario needs its own guardrail and eval case.

What I would do next

Move profile loading into a pre-run hook instead of a Step 1 tool call. Add SQLiteSession and multi-turn eval cases for continuation and correction. Add model-specific eval baselines so a model downgrade is caught automatically. Add at least two more named guardrails: SUSPENDED_STUDENT and REGISTRATION_DEADLINE.